

Reviewer Pack — Minimal Geometric Entropy of Boolean Functions vs. Resolutio...

Entry 59bb76fe0642 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 11:25:05 UTC

Minimal Geometric Entropy of Boolean Functions vs. Resolution Proof Width

Entry ID: 59bb76fe0642 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 11:25:05 UTC

1. Conjecture

Field A (mathematical branch): Geometric Measure Theory (Fractal Dimension) **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

Statement:

For any given Boolean function f with n inputs, the resolution proof width $w(f)$ of f is linearly correlated with the minimal geometric entropy $H_{\min}(G_f)$ of its associated induced graph G_f , such that $w(f) = \Theta(H_{\min}(G_f))$.

Rationale (proposer's reasoning):

Geometric entropy measures the complexity of a set by considering the minimum number of balls needed to cover the set. This measure could potentially capture the complexity inherent in Boolean functions that are reflected in their resolution proof complexity.

Taxonomy category: geometric_entropy_vs_resolution_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f4eceddd19266c235

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all generated Boolean functions with n inputs, the correlation coefficient between resolution proof width $w(f)$ and minimal geometric entropy $H_{\min}(G_f)$ is ≥ 0.8 within a time limit of T seconds, and no seed produces a correlation coefficient < 0.2 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 11 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - geometric measure theory boolean satisfiability resolution proof complexity - induced graph Boolean function minimal geometric entropy resolution width - correlation fractal dimension Boolean Satisfiability resolution proof width

Top relevant hits considered: - [http://arxiv.org/abs/1103.5740v2] Generating and Searching Families of FFT Algorithms - [http://arxiv.org/abs/1611.00827v2] Geometric complexity theory and matrix powering - [http://arxiv.org/abs/1602.06867v4] Lower bound for the Complexity of the Boolean Satisfiability Problem - [http://arxiv.org/abs/2003.09703v1] Variance function of boolean additive convolution - [http://arxiv.org/abs/1505.04658v2] A survey of recent results in (generalized) graph entropies - [http://arxiv.org/abs/1408.5939v2] Planar Induced Subgraphs of Sparse Graphs - [http://arxiv.org/abs/1603.09409v4] Minkowski dimension and explicit tube formulas for p -adic fractal strings - [http://arxiv.org/abs/1501.04911v2] Fractal dimension and turbulence in Giant HII Regions

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.2s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
from itertools import combinations

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_geometric_entropy(G):
        n = len(G)
        if n == 0:
            return 0
        max_radius = math.ceil(math.log2(n))
        min_balls = float('inf')
        for radius in range(1, max_radius + 1):
            balls = set()
            for node in G:
                ball = {node}
                queue = [node]
                while queue:
                    current = queue.pop(0)
```

```

        if current not in balls:
            balls.add(current)
        for neighbor in G[current]:
            if neighbor not in balls and abs(neighbor - current) <= radius:
                queue.append(neighbor)
    min_balls = min(min_balls, len(balls))
    return math.log2(min_balls)

def compute_resolution_width(f):
    n = int(math.log2(len(f)))
    clauses = []
    for i in range(n):
        for j in range(i + 1, n):
            if f[2**i] != f[2**j]:
                clauses.append([i, -j])
                clauses.append([-i, j])
    model = {}

    def dpll(model, clauses):
        unit_clause = next((c for c in clauses if len(c) == 1), None)
        if unit_clause:
            literal = unit_clause[0]
            if literal < 0:
                literal = -literal
            model[literal] = False
        else:
            model[literal] = True
        new_clauses = []
        for c in clauses:
            if literal not in c and -literal not in c:
                new_clauses.append(c)
            elif literal in c:
                continue
            else:
                new_c = [x for x in c if x != -literal]
                new_clauses.append(new_c)
        return dpll(model, new_clauses)
    unsatisfied_clauses = [c for c in clauses if not any(literal in model and model[literal] for literal in c)]
    if not unsatisfied_clauses:
        return True
    literal = unsatisfied_clauses[0][0]
    new_model_true = {**model, literal: True}
    if dpll(new_model_true, clauses):
        return True
    new_model_false = {**model, literal: False}
    return dpll(new_model_false, clauses)

width = 1
while not dpll(model, clauses):
    model[literal] = True
    width += 1
return width

n_values = [5, 10, 15, 20, 30, 40]
results = []

```

```

for n in n_values:
    instances_tested = 0
    total_width = 0
    total_entropy = 0

    while len(results) < 30:
        f = generate_boolean_function(n)
        G = {i: set() for i in range(2**n)}
        for i in range(2**n):
            for j in range(i + 1, 2**n):
                if f[i] != f[j]:
                    G[i].add(j)
                    G[j].add(i)

        width = compute_resolution_width(f)
        entropy = compute_geometric_entropy(G)

        instances_tested += 1
        total_width += width
        total_entropy += entropy

        results.append({
            "metric_name": "resolution_width",
            "metric_value": width,
            "instances_tested": instances_tested,
            "n_max": n,
            "conjecture_holds": None,
            "counterexample": ""
        })

    mean_width = total_width / len(results)
    std_width = math.sqrt(sum((x['metric_value'] - mean_width) ** 2 for x in results) / len(results))

    return {
        "seed": seed,
        "mean_resolution_width": mean_width,
        "std_resolution_width": std_width,
        "instances_tested": len(results),
        "n_max": max(x['n_max'] for x in results),
        "conjecture_holds": None,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_width = sum(x['mean_resolution_width'] for x in results) / len(results)
    std_width = math.sqrt(sum((x['mean_resolution_width'] - mean_width) ** 2 for x in results) / len(results))
    support_fraction = sum(1 for x in results if x['conjecture_holds']) / len(results)

```

```

if all(x['conjecture_holds'] is True for x in results):
    print(f"RESULT: SUPPORTED mean={mean_width} std={std_width} support_fraction={support_fraction}")
elif any(x['conjecture_holds'] is False for x in results):
    first_failing_seed = next(i for i, x in enumerate(results) if x['conjecture_holds'] is False)
    print(f"RESULT: FALSIFIED counterexample=\"first failing seed\" first_failing_seed={seeds[first_failing_seed]}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
L: {'seed': 421, 'mean_resolution_width': 0.0, 'std_resolution_width': 1.0, 'instances_tested': 30,
TRIAL: {'seed': 463, 'mean_resolution_width': 0.0, 'std_resolution_width': 1.0, 'instances_tested': 30,
TRIAL: {'seed': 503, 'mean_resolution_width': 0.0, 'std_resolution_width': 1.0, 'instances_tested': 30,
TRIAL: {'seed': 547, 'mean_resolution_width': 0.0, 'std_resolution_width': 1.0, 'instances_tested': 30,
TRIAL: {'seed': 593, 'mean_resolution_width': 0.0, 'std_resolution_width': 1.0, 'instances_tested': 30,
TRIAL: {'seed': 631, 'mean_resolution_width': 0.0, 'std_resolution_width': 1.0, 'instances_tested': 30,
TRIAL: {'seed': 677, 'mean_resolution_width': 0.0, 'std_resolution_width': 1.0, 'instances_tested': 30,
TRIAL: {'seed': 727, 'mean_resolution_width': 0.0, 'std_resolution_width': 1.0, 'instances_tested': 30,
TRIAL: {'seed': 773, 'mean_resolution_width': 0.0, 'std_resolution_width': 1.0, 'instances_tested': 30,
TRIAL: {'seed': 821, 'mean_resolution_width': 0.0, 'std_resolution_width': 1.0, 'instances_tested': 30,
TRIAL: {'seed': 877, 'mean_resolution_width': 0.0, 'std_resolution_width': 1.0, 'instances_tested': 30,
TRIAL: {'seed': 929, 'mean_resolution_width': 0.0, 'std_resolution_width': 1.0, 'instances_tested': 30,
RESULT: INCONCLUSIVE reason=unknown
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code provided is incomplete and does not contain the full implementation of the trial loop, which is necessary to evaluate the conjecture. Additionally, the computation of geometric entropy seems flawed as it does not account for all possible balls and their radii properly.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code provided is incomplete and does not contain the full implementation of the trial loop, which is necessary to evaluate the conjecture. Additionally, the computation of geometric entropy seems flawed as it does not account for all possible balls and their radii properly. | next: Complete the test code with a proper implementation of the trial loop and correct the computation of geometric entropy. Then retest the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13436
2	propose	ollama_remote	glm4:latest	0	0	12016
3	propose	ollama_remote	glm4:latest	0	0	17355
4	preregistration	ollama_remote	glm4:latest	0	0	13325
5	novelty	ollama_remote	glm4:latest	0	0	8274
6	novelty	ollama_remote	glm4:latest	0	0	18452
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21526
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12981
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14948
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14969
11	critic	ollama_remote	glm4:latest	0	0	17429
12	judge	ollama_remote	glm4:latest	0	0	9306

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 174017 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/59bb76fe0642.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/59bb76fe0642.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/59bb76fe0642.tar.gz` (if generated)